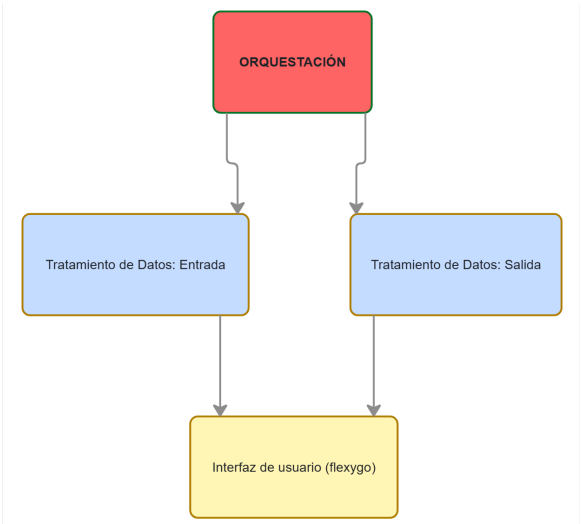
AHORA BRAIN – RESUMEN

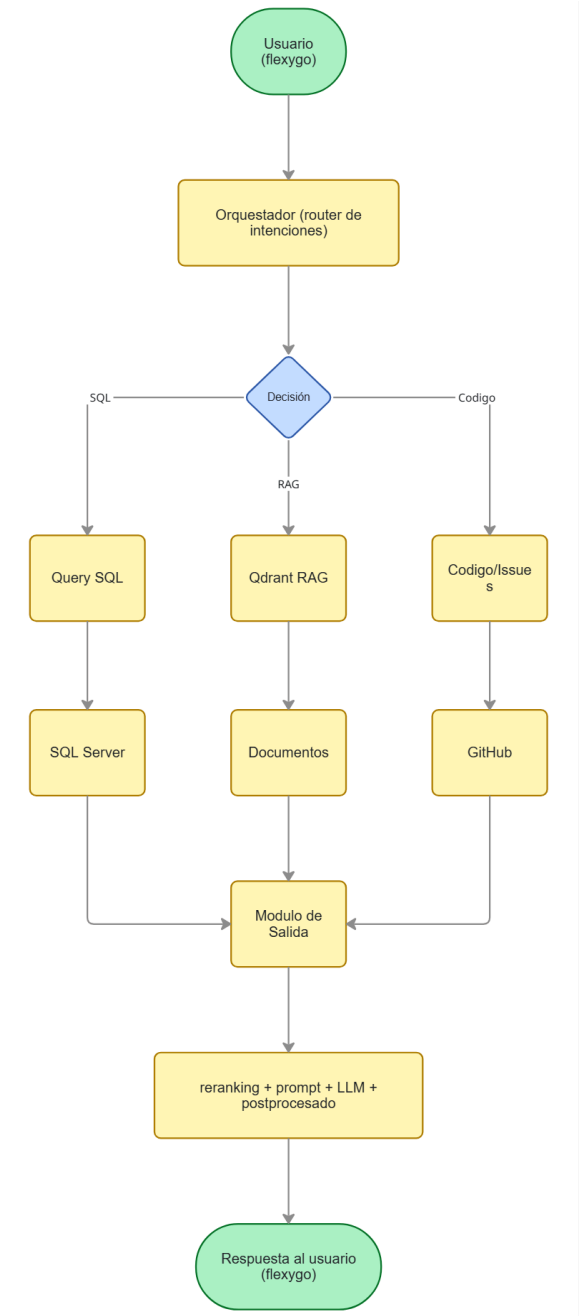
1. DESCRIPCIÓN GENERAL

AHORA Brain es un sistema de Inteligencia Artificial centralizado para la empresa AHORA Software. El objetivo es construir una capa de inteligencia unificada que actúa como cerebro de la organización: capaz de consultar datos estructurados del ERP/CRM, documentación técnica y repositorios de código, y devolver respuestas en lenguaje natural a través de una interfaz conversacional.

El sistema se basa en una arquitectura RAG (Retrieval Augmented Generation) evolucionada hacia un modelo agéntico, donde la IA no solo recupera información sino que razona sobre qué fuente y que herramienta utilizar en función de la consulta recibida.

2. ARQUITECTURA GENERAL DEL SISTEMA





3. FUENTES DE DATOS

El sistema consume información de tres orígenes principales:

1. SQL Server 2025 (datos estructurados)
- ERP: ventas, facturación, contratos, rentabilidad, horas imputadas
- CRM: clientes, partners, oportunidades
- SQL Server 2025 incluye capacidades nativas de vectorización, lo que permite consultas semánticas directamente sobre la base de datos relacional
- Para este tipo de datos se usará Text-to-SQL con una capa semántica que define métricas de negocio (ventas netas, churn rate, avance de proyecto) de forma determinista para evitar alucinaciones en cálculos financieros

2. Documentos no estructurados (tipo aún por confirmar)
- Posibles formatos: PDF, Excel, Word, presentaciones, manuales
- Fuentes: disco duro SSD corporativo, Notebook, Freshdesk (tickets)
- Para este tipo de datos se usará Quadrant como base de datos vectorial
- El proceso implica: transformación del documento -> chunking semántico -> generación de embeddings -> indexado en Quadrant
- Los embeddings se generan con el modelo nomic-embed-text via Ollama

3. GitHub (repositorios de código - pendiente de confirmar)
- Código fuente C# / .NET Core
- Issues, merge requests, comentarios de desarrollo
- Notas de versión, roadmap de producto
- Se indexara en Quadrant igual que los documentos pero con prompts especializados para respuestas tecnicas de desarrollo

STACK TECNOLÓGICO

Orquestacion:
- n8n (self-hosted): orquestador principal de todos los flujos
- Modelo LLM router: decide a qué rama enrutar cada consulta

Base de datos vectorial:
- Quadrant: almacenamiento de embeddings para documentos no estructurados

Bases de datos:
- SQL Server 2025: datos estructurados ERP/CRM

Modelos de IA:
- Ollama (local): ejecucion de modelos en el servidor propio
- Modelo de chat: qwen2.5 (pruebas), Llama 4 o similar (produccion)

- Modelo de embeddings: nomic-embed-text
- Reranker: Cohere rank-multilenguaje-v3.0 (via API)

Infraestructura:
- Servidor local con GPU (privacidad total, coste marginal cero)
- Opción alternativa: Cloud GPU VPS (RumPed, Lambda Labs)

Interfaz de usuario:
- FlexyGo (plataforma low-code propia de AHORA)
- Usado tanto para la carga de datos como para el chat con la IA
- Se integra directamente con n8n vía webhooks

Integraciones ya operativas:
- Dos flujos en n8n que extraen tickets de Freshdesk y los cargan en flexygo
- Flujo de prueba Quadrant funcional en n8n

4. FLUJO DE DATOS: DE LA FUENTE A LA RESPUESTA

El procesamiento de datos sigue este orden:

PASO 1 - Transformación
Los datos brutos se limpian, normalizan y convierten al formato adecuado.
Para documentos: extracción de texto, conversión a Markdown, limpieza.
Para SQL: definición de la capa semántica con métricas de negocio.

PASO 2 - Vectorización (solo datos no estructurados)
Los documentos transformados se dividen en chunks semánticos.
Cada chunk lleva breadcrumbs contextuales (ej: Empresa > Soporte > Manual).
Se generan embeddings con nomic-embed-text y se indexan en Quadrant.

PASO 3 - Recuperación (en tiempo de consulta)
El orquestador recibe la pregunta del usuario y decide la rama.
Para SQL: genera una query validada por la capa semántica.
Para documentos: búsqueda por similitud en Quadrant.
Para código: busqueda en Quadrant con filtros de tipo de archivo.

PASO 4 - Reranking (solo rama documentos)
Los chunks recuperados de Quadrant se ordenan por relevancia real usando Cohere Rank, que va más allá de la similitud vectorial.

PASO 5 - Construcción del prompt
Se construye un prompt específico según el tipo de fuente:
- Prompt SQL: incluye datos tabulares, prohíbe inventar cifras
- Prompt documentos: incluye chunks con citas de fuente numeradas
- Prompt codigo: incluye bloques de código con contexto tecnico

PASO 6 - Generación de respuesta
El LLM (llamado local) genera la respuesta usando el prompt construido.

PASO 7 - Post-procesamiento
La respuesta se formatea para la UI: tablas Markdown, citas de fuente, formato de código. Se genera metadata de evaluación para métricas RAG.

PASO 8 - Entrega al usuario
La respuesta llega a flexygo con las fuentes utilizadas y el tipo de consulta realizada.